ЛР2. СКУД: диаграммы классов, объектов и последовательностей, код на C#

Все схемы лежат в общем файле лабораторных работ СКУД.drawio на страницах «ЛР2 — …», по одной на странице: две диаграммы классов, диаграмма объектов и три диаграммы последовательностей. Программный код находится в файле Skud.cs. Тринадцать классов системы не помещаются в одну читаемую схему, поэтому диаграмма классов разделена на две.

Диаграмма классов 1 из 2: пропуска, зоны и двери

ЛР2. Диаграмма классов СКУД (1 из 2): пропуска, зоны и двери

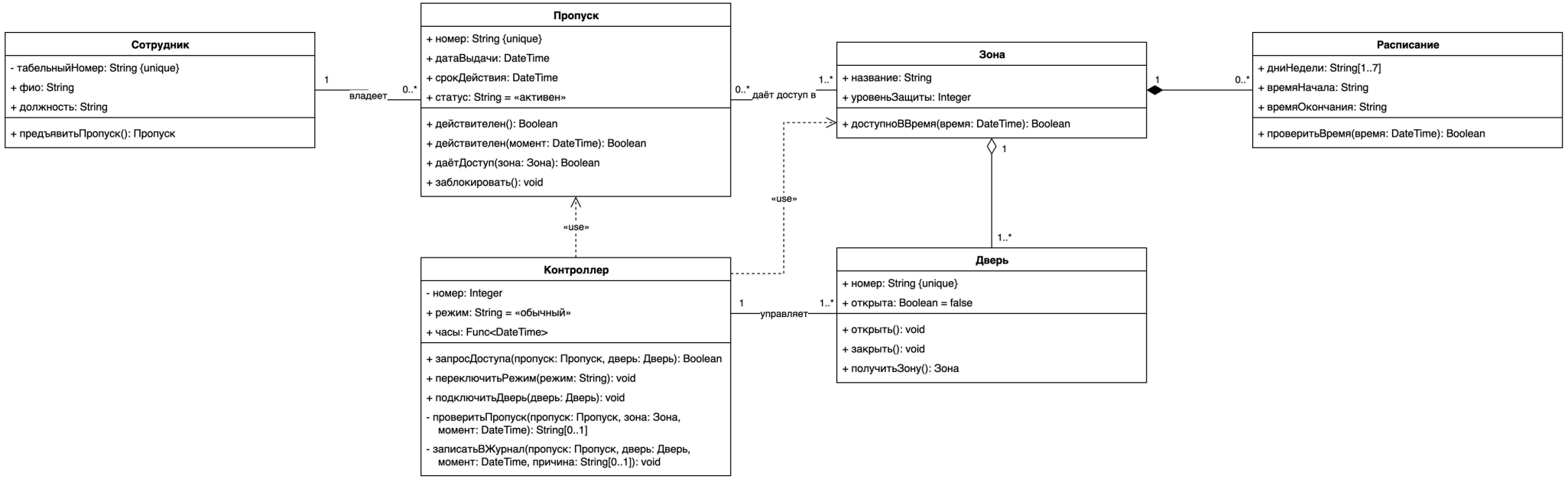


Диаграмма классов 1 из 2: пропуска, зоны и двери

Вопрос	Какие классы определяют, кто, когда и через какую дверь может пройти в зону?
Читатель	Разработчик.
Что сделает читатель	Реализует классы Сотрудник, Пропуск, Зона, Расписание, Дверь и Контроллер с перечисленными полями и методами и выразит связи между ними полями-коллекциями.
Что показано на схеме	1. Цепочка Сотрудник — Пропуск — Зона: право прохода принадлежит пропуску, поэтому права можно менять, не изменяя данные сотрудника. 2. Агрегация Зона ◇ Дверь и композиция Зона ◆ Расписание: виды связей различаются, потому что дверь является самостоятельным оборудованием, а расписание без зоны не имеет смысла; это определяет, кто создаёт объект. 3. Пунктирные зависимости Контроллера от Пропуска и Зоны: объекты нужны только во время запроса и не хранятся в полях, поэтому разработчик не должен добавлять для них поля. 4. Закрытые методы `проверитьПропуск` и `записатьВЖурнал` и атрибут `часы` в Контроллере: разделяют запрос доступа на проверку и запись и выносят источник времени, что позволяет тестировать без системных часов.
Что не показано	1. Где хранятся пропуска и права и как контроллер находит пропуск по считанному коду. 2. Как пропуска выдаются, блокируются и назначаются зонам. 3. Как ведётся журнал и что происходит при аварии: эти вопросы закрывает вторая диаграмма классов. 4. В каком порядке выполняются проверки внутри запроса доступа: этот вопрос закрывает диаграмма последовательностей 1.
Допущения	1. Кратности `0..*` и `1..*` в коде выражены коллекциями `List`, ограничение числа элементов типом не задаётся. 2. Ограничение `{unique}` на номерах и кратность `String[1..7]` у атрибута `дниНедели` язык не проверяет. 3. Различие между агрегацией и композицией в C# не выражается: обе связи реализуются полем-ссылкой, поэтому «расписание не существует без зоны» обеспечивается только тем, что расписание создаётся методом `Зона.добавитьРасписание`. 4. Ассоциации показаны линией без направления, а в коде ссылка хранится только в одном классе, например список дверей в Контроллере, обратной ссылки на контроллер в Двери нет. 5. Зависимость в C# не объявляется явно и выражается только использованием типа в параметре или локальной переменной.

Диаграмма классов 2 из 2: считыватели, журнал и аварийный режим

ЛР2. Диаграмма классов СКУД (2 из 2): считыватели, журнал и аварийный режим

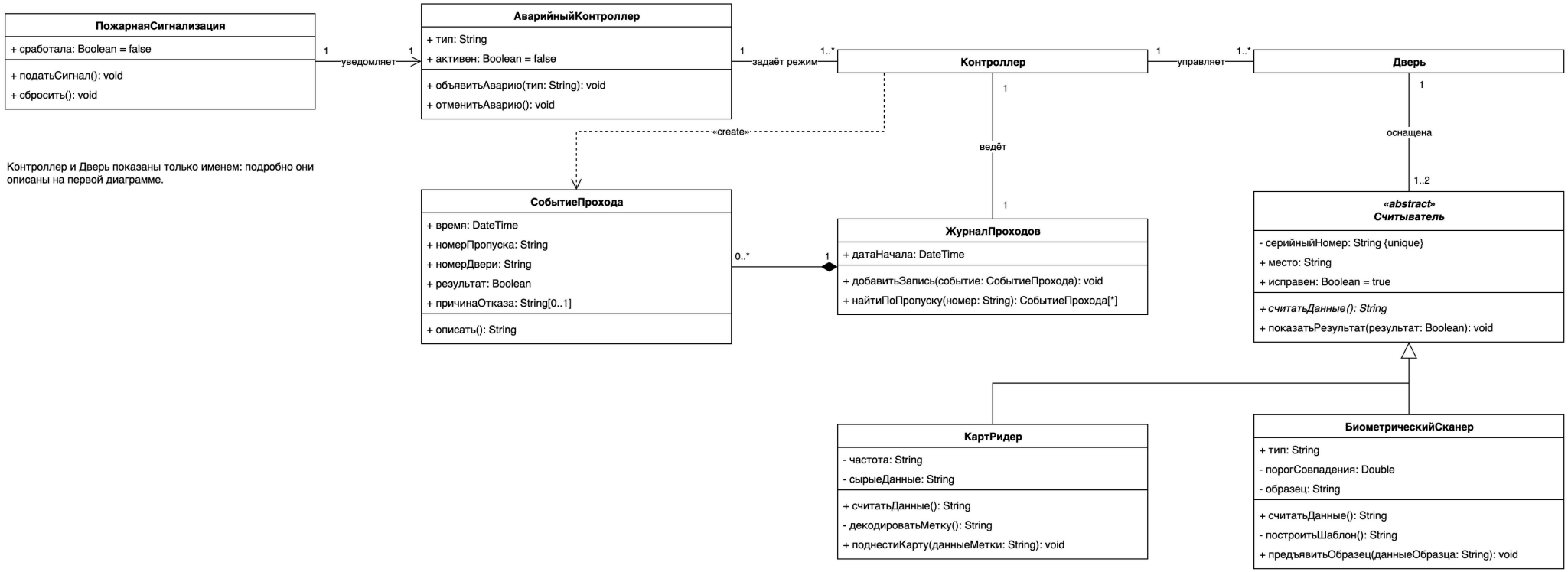
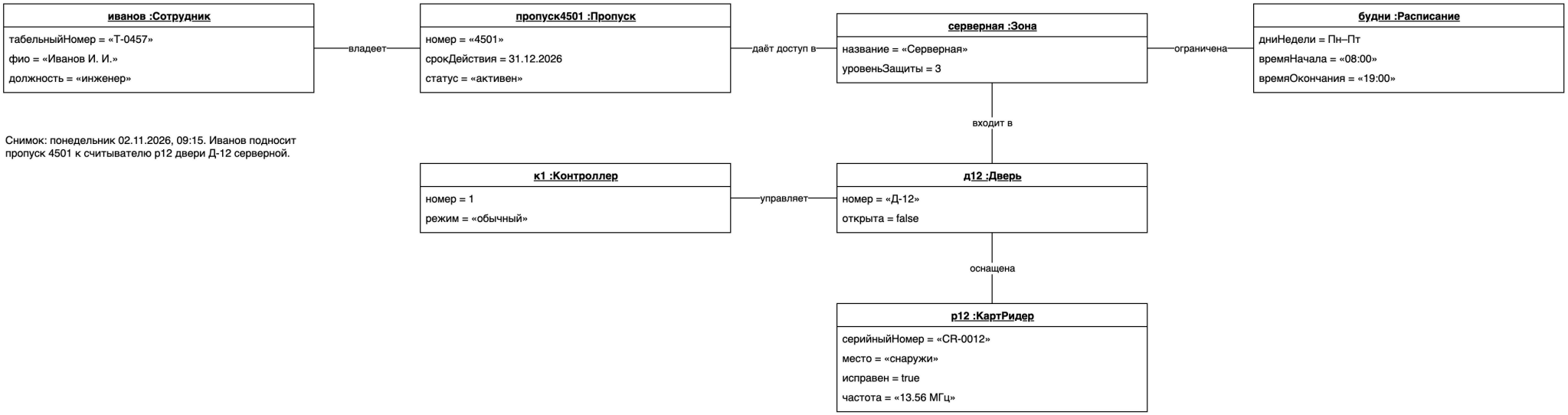


Диаграмма классов 2 из 2: считыватели, журнал и аварийный режим

Вопрос	Из каких частей состоит оборудование СКУД и как фиксируются проходы и аварии?
Читатель	Разработчик.
Что сделает читатель	Добавит новый тип считывателя наследованием от класса Считыватель с реализацией метода `считатьДанные` и подключит его к двери.
Что показано на схеме	1. Абстрактный класс Считыватель и два потомка, КартРидер и БиометрическийСканер: это единственная точка расширения оборудования, новый способ идентификации добавляется потомком без изменения Контроллера и Двери. 2. Композиция ЖурналПроходов ◆ СобытиеПрохода: запись имеет смысл только внутри журнала, поэтому журнал отвечает за хранение, а событие за описание прохода. 3. Зависимость «create» Контроллера от СобытиеПрохода: ответственность за создание записи закреплена за контроллером, а не за журналом или дверью. 4. Цепочка ПожарнаяСигнализация → АварийныйКонтроллер → Контроллер: аварийный режим включается отдельным объектом, что отделяет аварийную логику от обычной. 5. Контроллер и Дверь повторены только именем: связи с ними видны без повторного описания классов.
Что не показано	1. Где и как долго хранится журнал: в памяти, в базе данных или в файле. 2. Как считыватель физически передаёт данные контроллеру. 3. Откуда поступает сигнал пожарной сигнализации: от датчика или из внешней системы. 4. Какие аварии, кроме пожара, поддерживает система.
Допущения	1. Кратность «1..2 считывателя» у двери типом не выражается, в коде она проверяется в методе `Дверь.добавитьСчитыватель` с выбросом исключения. 2. Композиция ЖурналПроходов ◆ СобытиеПрохода не запрещает создать запись вне журнала: конструктор записи открыт, так как её создаёт контроллер. 3. Ассоциации показаны линией без направления, а в коде ссылка хранится только в одном классе, например список считывателей в Двери, обратной ссылки на дверь в Считывателе нет. 4. Зависимость «create» в коде является обычным вызовом конструктора внутри метода.

Диаграмма объектов: проход сотрудника Иванова

ЛР2. Диаграмма объектов СКУД



Снимок: понедельник 02.11.2026, 09:15. Иванов подносит пропуск 4501 к считывателю р12 двери Д-12 серверной.

Диаграмма объектов: проход сотрудника Иванова

Вопрос	Какие объекты и с какими значениями участвуют в проходе сотрудника Иванова через дверь Д-12 в понедельник 02.11.2026 в 09:15?
Читатель	Тестировщик.
Что сделает читатель	Создаст в тесте те же объекты с теми же значениями и проверит, что запрос доступа в этом состоянии разрешается.
Что показано на схеме	1. Семь объектов с конкретными значениями атрибутов: значений достаточно, чтобы воспроизвести сценарий в тесте без дополнительных данных. 2. Связи от сотрудника до считывателя: показывают путь, по которому проверка доступа доходит от человека до двери. 3. Момент перед проходом, когда дверь закрыта, режим контроллера обычный, а время попадает в расписание: в этом состоянии результат определяется только данными, поэтому тест однозначен.
Что не показано	1. Откуда берутся объекты: из хранилища или создаются при запуске. 2. Что происходит после запроса: открытие двери и запись в журнал. 3. Как выглядит то же состояние при отказе или при аварии. 4. Другие сотрудники, двери и зоны системы.
Допущения	1. Подписи связей «владеет», «даёт доступ в», «ограничена», «входит в», «оснащена» и «управляет» в коде отсутствуют, связи реализованы полями и коллекциями. 2. Имена объектов `иванов`, `д12` и другие в коде являются локальными переменными. 3. Связь двери с зоной показана одной линией, а в коде это две ссылки: список дверей в Зоне и поле зоны в Двери. 4. Диаграмма показывает состояние в один момент, а в коде такое состояние получается последовательностью создающих вызовов.

Диаграмма последовательностей 1 из 3: проход через дверь

ЛР2. Диаграмма последовательностей СКУД (1 из 3): проход через дверь

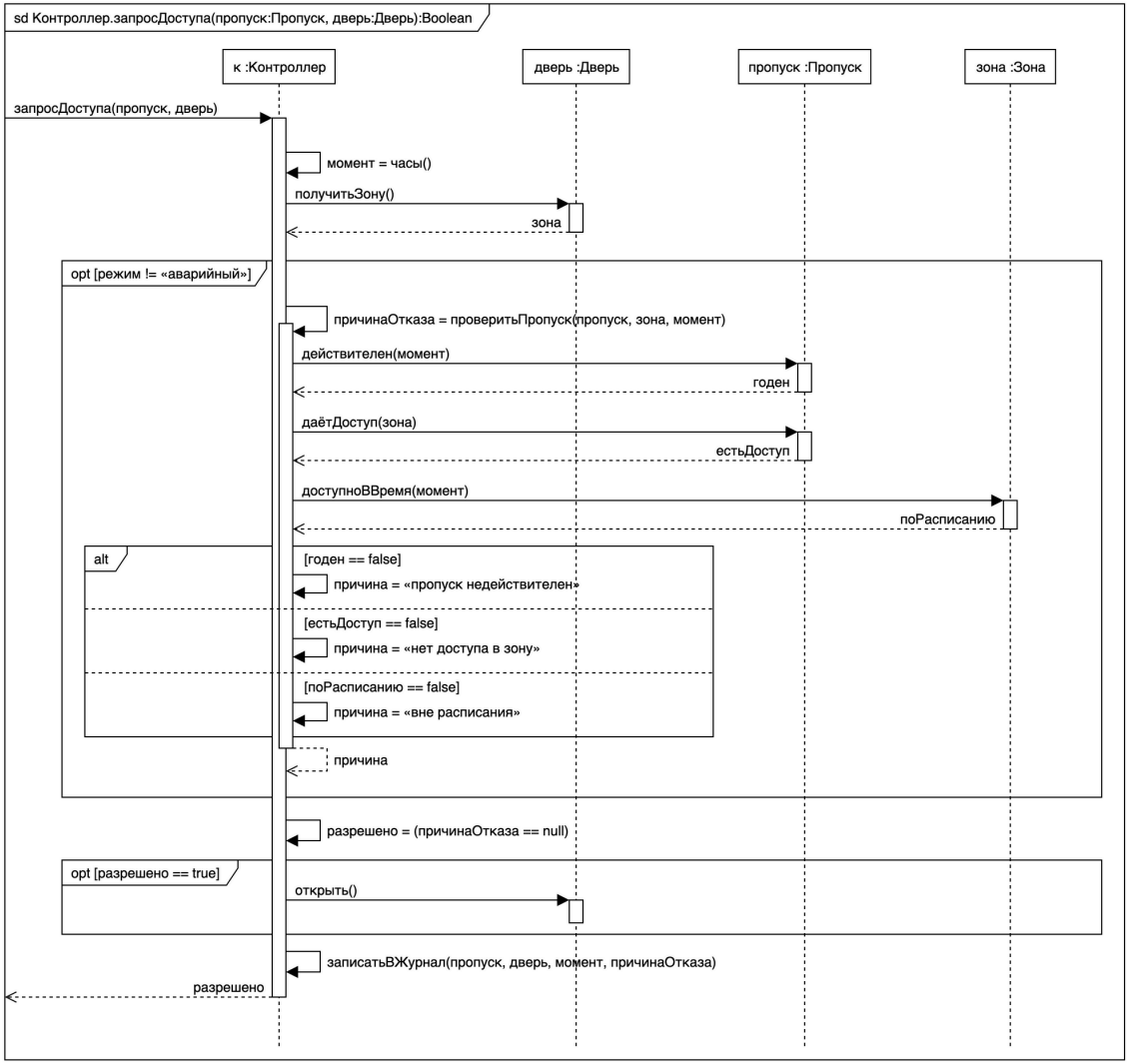


Диаграмма последовательностей 1 из 3: проход через дверь

Вопрос	В каком порядке контроллер решает, пропустить ли сотрудника через дверь?
Читатель	Тестировщик.
Что сделает читатель	Составит тесты по ветвям проверки: пропуск недействителен, нет доступа в зону, вне расписания, проход разрешён.
Что показано на схеме	1. Операция `Контроллер.запросДоступа` с четырьмя участниками: решение принимает контроллер, а пропуск, зона и дверь лишь отвечают на его вызовы. 2. Условие `opt [режим != «аварийный»]`: в аварийном режиме проверки обходятся, поэтому этот случай тестируется отдельно. 3. Вложенная активность `проверитьПропуск`: три запроса и выбор причины объединены в одну операцию, а фрагмент `alt` задаёт приоритет причин отказа. 4. Условие `opt [разрешено == true]`: дверь открывается только при отсутствии причины отказа. 5. Ответы подписаны именами переменных вызывающей стороны (`зона`, `годен`, `естьДоступ`, `поРасписанию`): по диаграмме можно написать код без догадок.
Что не показано	1. Откуда приходит запрос доступа и как данные пропуска попадают к контроллеру. 2. Что происходит после отказа: сигнал, повторная попытка. 3. Как и когда дверь закрывается после прохода. 4. Как формируется запись журнала: это закрывает диаграмма последовательностей 3. 5. Как включается аварийный режим: это закрывает диаграмма последовательностей 2.
Допущения	1. Присваивания внутри `alt` показаны сообщениями контроллера самому себе, в коде это присваивания локальной переменной, а не вызовы. 2. Вызов `часы()` показан как самовызов, в коде это вызов делегата `Func<DateTime>`. 3. Ответ `проверитьПропуск` подписан «причина» по имени переменной внутри вызываемого метода, а вызывающий код сохраняет его в переменную `причинаОтказа`. 4. Вызывающая сторона не показана, поэтому значение `разрешено` возвращается в точку вне диаграммы.

Диаграмма последовательностей 2 из 3: пожар

ЛР2. Диаграмма последовательностей СКУД (2 из 3): пожар

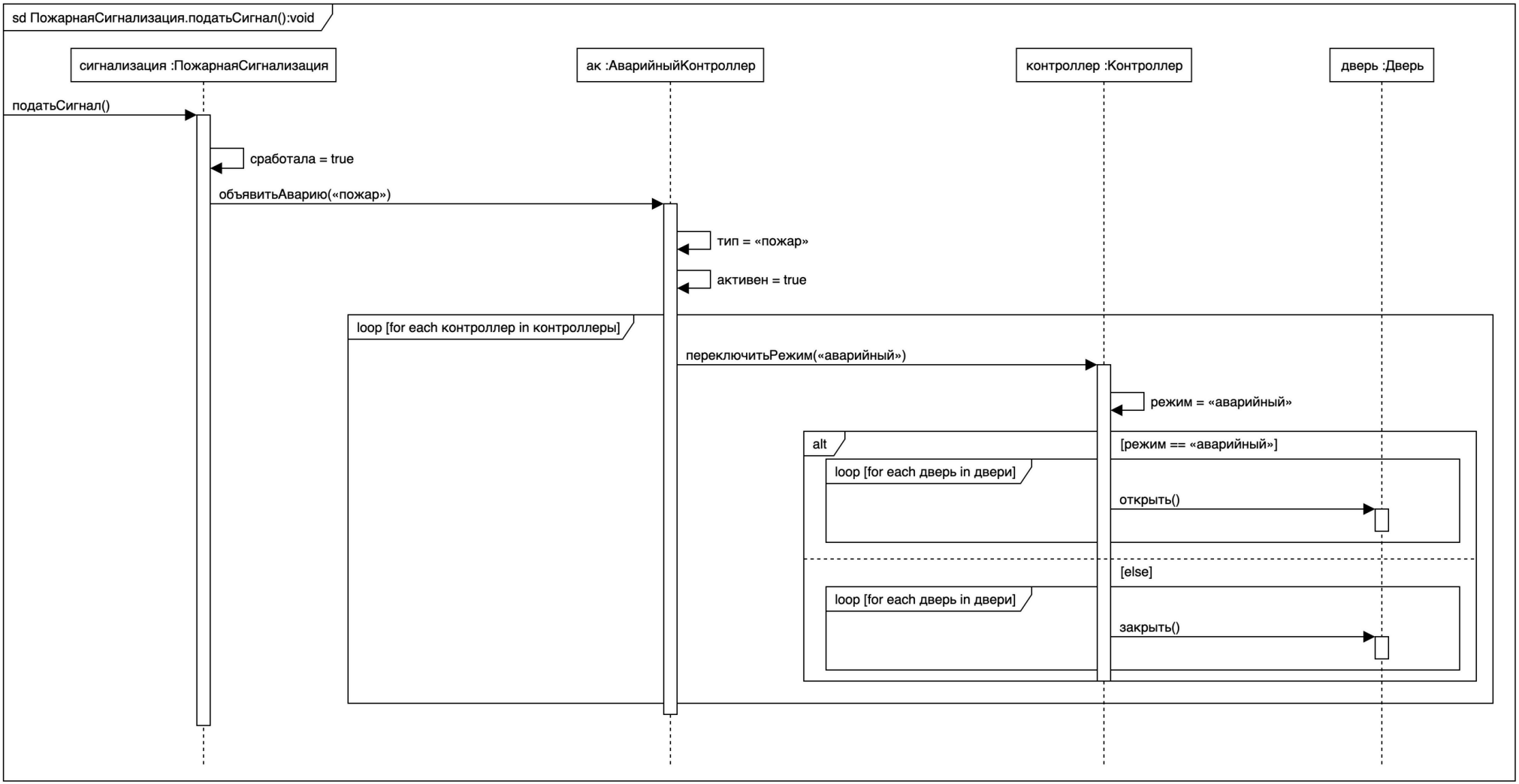


Диаграмма последовательностей 2 из 3: пожар

Вопрос	Как сигнал пожарной сигнализации приводит к открытию всех дверей?
Читатель	Менеджер.
Что сделает читатель	Подтвердит с заказчиком правило «при пожаре открываются все двери всех контроллеров» и условия возврата в обычный режим.
Что показано на схеме	1. Четыре участника и цепочка вложенных вызовов `податьСигнал`, `объявитьАварию`, `переключитьРежим`, `открыть`: каждый уровень отвечает за свой шаг, и по цепочке видно, на каком шаге применяется правило. 2. Цикл `loop` по контроллерам в аварийном контроллере: правило относится ко всем контроллерам, а не к одному, это существенное требование безопасности. 3. Условие `alt` в `переключитьРежим`: одна операция обслуживает и включение, и отмену аварии. 4. Внутренний цикл по дверям: контроллер открывает все свои двери без проверки пропусков.
Что не показано	1. Кто и когда отменяет аварию и нужно ли подтверждение. 2. Как обрабатываются запросы доступа в аварийном режиме и что при этом пишется в журнал. 3. Как сигнал поступает от реального датчика. 4. Что происходит при отказе датчика или потере связи с контроллером.
Допущения	1. Ветка `else` фрагмента `alt` показана для полноты, но в этом сценарии не выполняется. 2. Присваивания `сработала = true`, `тип = «пожар»`, `активен = true`, `режим = «аварийный»` показаны самовызовами, в коде это присваивания свойств с закрытым сеттером. 3. Условия фрагментов `loop [for each ...]` записаны псевдокодом, в коде они реализованы оператором `foreach`.

Диаграмма последовательностей 3 из 3: запись в журнал

ЛР2. Диаграмма последовательностей СКУД (3 из 3): запись в журнал

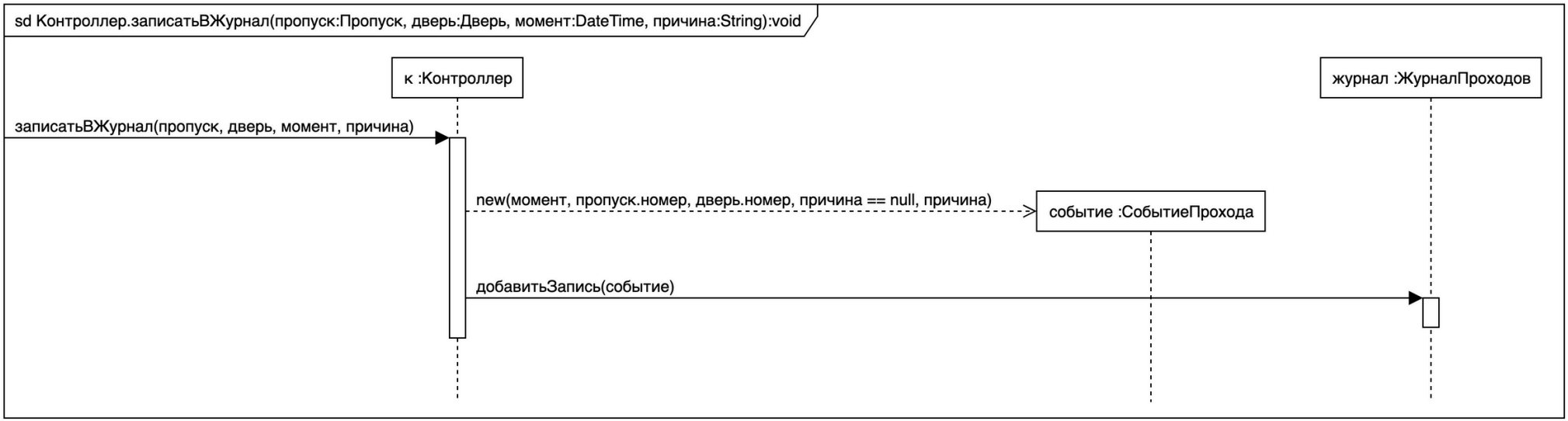


Диаграмма последовательностей 3 из 3: запись в журнал

Вопрос	Как создаётся запись о проходе и попадает в журнал?
Читатель	Разработчик.
Что сделает читатель	Реализует метод `Контроллер.записатьВЖурнал`: создаст объект СобытиеПрохода конструктором и передаст его журналу.
Что показано на схеме	1. Три участника: контроллер, создаваемое событие и журнал: это минимальный набор для показа создания объекта и его передачи. 2. Стрелка `new` к заголовку участника «событие»: объект появляется в момент вызова, то есть запись создаётся заново при каждом проходе. 3. Источники аргументов конструктора: номера пропуска и двери берутся из параметров, результат вычисляется как `причина == null`; событие хранит номера, а не ссылки на пропуск и дверь. 4. Вызов `добавитьЗапись`: журнал принимает уже готовую запись и сам её не формирует.
Что не показано	1. Где и как журнал хранит записи. 2. Что происходит при ошибке записи. 3. Как записи читаются и как долго хранятся. 4. Как проверяется доступ: это закрывает диаграмма последовательностей 1.
Допущения	1. Стрелка `new` соответствует вызову конструктора C#, в UML у класса нет операции с таким именем. 2. Участник «событие» до создания не существует, в коде ему соответствует локальная переменная. 3. Выражение `причина == null` в аргументах показано как часть сообщения, в коде оно вычисляется до вызова конструктора.

Программный код

В файле Skud.cs реализованы те же 13 классов с теми же именами, атрибутами и методами. Связи диаграмм выражены полями и коллекциями. Порядок вызовов в методах `Контроллер.запросДоступа`, `Контроллер.записатьВЖурнал`, `Контроллер.переключитьРежим`, `ПожарнаяСигнализация.податьСигнал` и `АварийныйКонтроллер.объявитьАварию` совпадает с диаграммами последовательностей. Метод `Program.Main` демонстрирует сценарии: успешный проход, отказ вне расписания, пожар и выборку из журнала.

Skud.cs

```
#nullable enable
using System;
using System.Collections.Generic;
using System.Linq;
using System.Security.Cryptography;
using System.Text;

namespace Skud;

public class Расписание
{
  private static readonly string[] краткиеДни = { "Вс", "Пн", "Вт", "Ср", "Чт", "Пт", "Сб" };

  public string[] дниНедели { get; set; }
  public string времяНачала { get; set; }
  public string времяОкончания { get; set; }

  public Расписание(string[] дниНедели, string времяНачала, string времяОкончания)
  {
    this.дниНедели = дниНедели;
    this.времяНачала = времяНачала;
    this.времяОкончания = времяОкончания;
  }

  public bool проверитьВремя(DateTime время)
  {
    if (!дниНедели.Contains(краткиеДни[(int)время.DayOfWeek]))
    {
      return false;
    }
    var t = время.TimeOfDay;
    return t >= TimeSpan.Parse(времяНачала) && t <= TimeSpan.Parse(времяОкончания);
  }
}

public class Зона
{
  public string название { get; set; }
  public int уровеньЗащиты { get; set; }
  private readonly List<Расписание> расписания = new();
  public List<Дверь> двери { get; } = new();

  public Зона(string название, int уровеньЗащиты)
  {
    this.название = название;
    this.уровеньЗащиты = уровеньЗащиты;
  }

  public Расписание добавитьРасписание(string[] дниНедели, string времяНачала, string времяОкончания)
  {
    var расписание = new Расписание(дниНедели, времяНачала, времяОкончания);
    расписания.Add(расписание);
    return расписание;
  }

  public bool доступноВВремя(DateTime время)
  {
    return расписания.Count == 0 || расписания.Any(p => p.проверитьВремя(время));
  }
}
```

```
public abstract class Считыватель
{
  private readonly string серийныйНомер;

  public string место { get; set; }
  public bool исправен { get; set; } = true;

  protected Считыватель(string серийныйНомер, string место)
  {
    this.серийныйНомер = серийныйНомер;
    this.место = место;
  }

  public abstract string считатьДанные();

  public void показатьРезультат(bool результат)
  {
    Console.WriteLine($"  [{место}] {(результат ? "доступ разрешён" : "доступ запрещён")}");
  }
}

public class КартРидер : Считыватель
{
  private readonly string частота;
  private string сырыеДанные = "";

  public КартРидер(string серийныйНомер, string место, string частота) : base(серийныйНомер, место)
  {
    this.частота = частота;
  }

  public void поднестиКарту(string данныеМетки)
  {
    сырыеДанные = данныеМетки;
  }

  public override string считатьДанные()
  {
    return декодироватьМетку();
  }

  private string декодироватьМетку()
  {
    return сырыеДанные.Trim().ToUpperInvariant();
  }
}

public class БиометрическийСканер : Считыватель
{
  private readonly double порогСовпадения;
  private string образец = "";

  public string тип { get; set; }

  public БиометрическийСканер(string серийныйНомер, string место, string тип, double порогСовпадения) :
base(серийныйНомер, место)
  {
    this.тип = тип;
    this.порогСовпадения = порогСовпадения;
  }

  public void предъявитьОбразец(string данныеОбразца)
  {
    образец = данныеОбразца;
  }

  public override string считатьДанные()
  {
    return построитьШаблон();
  }

  private string построитьШаблон()
  {
    var хеш = SHA256.HashData(Encoding.UTF8.GetBytes(образец));
    return Convert.ToHexString(хеш)[..16];
```

```
  }
}

public class Дверь
{
  private readonly List<Считыватель> считыватели = new();
  private readonly Зона зона;

  public string номер { get; set; }
  public bool открыта { get; private set; }

  public Дверь(string номер, Зона зона)
  {
    this.номер = номер;
    this.зона = зона;
    зона.двери.Add(this);
  }

  public void добавитьСчитыватель(Считыватель считыватель)
  {
    if (считыватели.Count >= 2)
    {
      throw new InvalidOperationException("У двери не может быть больше двух считывателей");
    }
    считыватели.Add(считыватель);
  }

  public void открыть()
  {
    открыта = true;
  }

  public void закрыть()
  {
    открыта = false;
  }

  public Зона получитьЗону()
  {
    return зона;
  }
}

public class Пропуск
{
  public string номер { get; set; }
  public DateTime датаВыдачи { get; set; }
  public DateTime срокДействия { get; set; }
  public string статус { get; set; } = "активен";
  public List<Зона> зоны { get; } = new();

  public Пропуск(string номер, DateTime датаВыдачи, DateTime срокДействия)
  {
    this.номер = номер;
    this.датаВыдачи = датаВыдачи;
    this.срокДействия = срокДействия;
  }

  public bool действителен()
  {
    return действителен(DateTime.Now);
  }

  public bool действителен(DateTime момент)
  {
    return статус == "активен" && момент >= датаВыдачи && момент <= срокДействия;
  }

  public bool даётДоступ(Зона зона)
  {
    return зоны.Contains(зона);
  }

  public void заблокировать()
  {
    статус = "заблокирован";
```

```
  }
}

public class Сотрудник
{
  private readonly string табельныйНомер;

  public string фио { get; set; }
  public string должность { get; set; }
  public List<Пропуск> пропуска { get; } = new();

  public Сотрудник(string табельныйНомер, string фио, string должность)
  {
    this.табельныйНомер = табельныйНомер;
    this.фио = фио;
    this.должность = должность;
  }

  public Пропуск? предъявитьПропуск()
  {
    return пропуска.FirstOrDefault();
  }
}

public class СобытиеПрохода
{
  public DateTime время { get; }
  public string номерПропуска { get; }
  public string номерДвери { get; }
  public bool результат { get; }
  public string? причинаОтказа { get; }

  public СобытиеПрохода(DateTime время, string номерПропуска, string номерДвери, bool результат, string?
причинаОтказа)
  {
    this.время = время;
    this.номерПропуска = номерПропуска;
    this.номерДвери = номерДвери;
    this.результат = результат;
    this.причинаОтказа = причинаОтказа;
  }

  public string описать()
  {
    var итог = результат ? "разрешён" : $"отказ ({причинаОтказа})";
    return $"{время:dd.MM.yyyy HH:mm} пропуск {номерПропуска}, дверь {номерДвери}: {итог}";
  }
}

public class ЖурналПроходов
{
  private readonly List<СобытиеПрохода> записи = new();

  public DateTime датаНачала { get; set; }

  public ЖурналПроходов(DateTime датаНачала)
  {
    this.датаНачала = датаНачала;
  }

  public void добавитьЗапись(СобытиеПрохода событие)
  {
    записи.Add(событие);
  }

  public List<СобытиеПрохода> найтиПоПропуску(string номер)
  {
    return записи.Where(з => з.номерПропуска == номер).ToList();
  }
}

public class Контроллер
{
  private readonly int номер;
  private readonly List<Дверь> двери = new();
  private readonly ЖурналПроходов журнал;
```

```
public string режим { get; private set; } = "обычный";
public Func<DateTime> часы { get; set; } = () => DateTime.Now;

public Контроллер(int номер, ЖурналПроходов журнал)
{
  this.номер = номер;
  this.журнал = журнал;
}

public void подключитьДверь(Дверь дверь)
{
  двери.Add(дверь);
}

public bool запросДоступа(Пропуск пропуск, Дверь дверь)
{
  var момент = часы();
  var зона = дверь.получитьЗону();
  string? причинаОтказа = null;

  if (режим != "аварийный")
  {
    причинаОтказа = проверитьПропуск(пропуск, зона, момент);
  }

  var разрешено = причинаОтказа == null;
  if (разрешено)
  {
    дверь.открыть();
  }
  записатьВЖурнал(пропуск, дверь, момент, причинаОтказа);
  return разрешено;
}

private string? проверитьПропуск(Пропуск пропуск, Зона зона, DateTime момент)
{
  var годен = пропуск.действителен(момент);
  var естьДоступ = пропуск.даётДоступ(зона);
  var поРасписанию = зона.доступноВВремя(момент);
  string? причина = null;

  if (!годен)
  {
    причина = "пропуск недействителен";
  }
  else if (!естьДоступ)
  {
    причина = "нет доступа в зону";
  }
  else if (!поРасписанию)
  {
    причина = "вне расписания";
  }
  return причина;
}

private void записатьВЖурнал(Пропуск пропуск, Дверь дверь, DateTime момент, string? причина)
{
  var событие = new СобытиеПрохода(момент, пропуск.номер, дверь.номер, причина == null, причина);
  журнал.добавитьЗапись(событие);
}

public void переключитьРежим(string режим)
{
  this.режим = режим;
  if (режим == "аварийный")
  {
    foreach (var дверь in двери)
    {
      дверь.открыть();
    }
  }
  else
  {
    foreach (var дверь in двери)
```

```
      {
        дверь.закрыть();
      }
    }
  }
}

public class АварийныйКонтроллер
{
  private readonly List<Контроллер> контроллеры;

  public string тип { get; private set; } = "";
  public bool активен { get; private set; }

  public АварийныйКонтроллер(List<Контроллер> контроллеры)
  {
    this.контроллеры = контроллеры;
  }

  public void объявитьАварию(string тип)
  {
    this.тип = тип;
    активен = true;
    foreach (var контроллер in контроллеры)
    {
      контроллер.переключитьРежим("аварийный");
    }
  }

  public void отменитьАварию()
  {
    активен = false;
    foreach (var контроллер in контроллеры)
    {
      контроллер.переключитьРежим("обычный");
    }
  }
}

public class ПожарнаяСигнализация
{
  private readonly АварийныйКонтроллер аварийныйКонтроллер;

  public bool сработала { get; private set; }

  public ПожарнаяСигнализация(АварийныйКонтроллер аварийныйКонтроллер)
  {
    this.аварийныйКонтроллер = аварийныйКонтроллер;
  }

  public void податьСигнал()
  {
    сработала = true;
    аварийныйКонтроллер.объявитьАварию("пожар");
  }

  public void сбросить()
  {
    сработала = false;
    аварийныйКонтроллер.отменитьАварию();
  }
}

public static class Program
{
  public static void Main()
  {
    var серверная = new Зона("Серверная", 3);
    серверная.добавитьРасписание(new[] { "Пн", "Вт", "Ср", "Чт", "Пт" }, "08:00", "19:00");

    var д12 = new Дверь("Д-12", серверная);
    var р12 = new КартРидер("CR-0012", "снаружи", "13.56 МГц");
    д12.добавитьСчитыватель(р12);

    var журнал = new ЖурналПроходов(new DateTime(2026, 11, 1));
    var к1 = new Контроллер(1, журнал);
```

```
    к1.подключитьДверь(д12);

    var иванов = new Сотрудник("Т-0457", "Иванов И. И.", "инженер");
    var пропуск4501 = new Пропуск("4501", new DateTime(2026, 1, 15), new DateTime(2026, 12, 31));
    пропуск4501.зоны.Add(серверная);
    иванов.пропуска.Add(пропуск4501);

    var базаПропусков = new Dictionary<string, Пропуск> { [пропуск4501.номер] = пропуск4501 };

    void пройти(DateTime момент)
    {
      к1.часы = () => момент;
      р12.поднестиКарту(иванов.предъявитьПропуск()?.номер ?? "");
      var пропуск = базаПропусков[р12.считатьДанные()];
      var разрешено = к1.запросДоступа(пропуск, д12);
      р12.показатьРезультат(разрешено);
      д12.закрыть();
    }

    Console.WriteLine("Понедельник, 09:15:");
    пройти(new DateTime(2026, 11, 2, 9, 15, 0));

    Console.WriteLine("Суббота, 10:00:");
    пройти(new DateTime(2026, 11, 7, 10, 0, 0));

    Console.WriteLine("Пожар:");
    var аварийный = new АварийныйКонтроллер(new List<Контроллер> { к1 });
    var сигнализация = new ПожарнаяСигнализация(аварийный);
    сигнализация.податьСигнал();
    Console.WriteLine($"  режим: {к1.режим}, дверь открыта: {д12.открыта}");
    сигнализация.сбросить();
    Console.WriteLine($"  режим: {к1.режим}, дверь открыта: {д12.открыта}");

    Console.WriteLine("Журнал по пропуску 4501:");
    foreach (var запись in журнал.найтиПоПропуску("4501"))
    {
      Console.WriteLine("  " + запись.описать());
    }
  }
}
```